Université du Québec à Chicoutimi

Département d'informatique et de mathématique

Rapport de Sprint 2

Cours : 8INF887 – Apprentissage profond

Hiver 2026 – Groupe 01

Projet :

Sous-titrage et traduction automatique de films/vidéos

Étudiant :

BEHAREL Armand

(Travail individuel)

Professeur : Kévin Bouchard, Ph.D.

Date de remise : 10 avril 2026

Table des matières

1 Travail réalisé **2**
1.1 2.1 - Implémentation du bloc Conformer 2
1.2 2.2 - Beam Search avec pénalité de longueur 3
1.3 2.4 - Entraînement et résultats préliminaires 4

2 Bilan du Sprint 2 **5**
2.1 Objectifs atteints 6
2.2 Difficultés rencontrées 6
2.3 Réflexion sur les résultats 6

3 Planification du Sprint final **6**

Travail réalisé

Ce sprint a pour objectif principal la migration de l'encodeur Transformer standard (Sprint 1) vers une architecture **Conformer**, ainsi que l'amélioration du décodage par **beam search** avec pénalité de longueur. L'entraînement est en cours sur le jeu de données Common Voice FR.

2.1 - Implémentation du bloc Conformer

Architecture mise à jour

Conformer Encoder (Gulati et al., 2020) – remplace le Transformer Encoder
Référence : *Conformer : Convolution-augmented Transformer for Speech Recognition*

Le passage au Conformer a été envisagé dès le Sprint 1. Un Transformer seul ne prend pas naturellement en compte la localité temporelle, pourtant essentielle pour des signaux audio composés de phonèmes courts. Le Conformer répond à cette limite en combinant, dans chaque couche, des convolutions locales et une attention globale.

Structure d'une couche Conformer :

```python
def forward(self, x, src_key_padding_mask=None):
    # FeedForward 1
    x = 0.5 * self.ff1(self.norm_ff1(x)) + x

    # Self-Attention
    residual = x
    x, _ = self.self_attn(self.norm_attn(x), ...)
    x = x + residual

    # Module de convolution (depthwise)
    x = self.conv_module(x)    # gere son propre residu

    # FeedForward
    x = 0.5 * self.ff2(self.norm_final(x)) + x

    return x
```

Listing 1 – Structure d'une couche ConformerLayer

Le module de convolution (`ConformerConvModule`) :

Le cœur du Conformer repose sur son module de convolution, qui suit la séquence suivante :

1. **LayerNorm** : normalisation de l'entrée
2. **Pointwise Conv** (1×1) : expansion $d_{model} -> 2, d_{model}$
3. **GLU** (*Gated Linear Unit*) : $\mathrm{GLU}(A, B) = A \odot \sigma(B)$, retour à d_{model}
4. **Depthwise Conv** (noyau 31, `groups =`, d_{model}) : convolution indépendante sur chaque canal
5. **BatchNorm** + **Swish** : $x \cdot \sigma(x)$

6. **Pointwise Conv** (1×1) : projection vers d_{model}
7. **Dropout** + connexion résiduelle

Pourquoi la convolution *depthwise* ?

Une convolution standard applique un noyau de taille $k \times C_{in} \times C_{out}$, ce qui mélange l'information entre tous les canaux.

La convolution *depthwise*, au contraire, applique un noyau de taille $k \times 1$ **par canal** (avec le paramètre `groups`$= C$). Elle permet ainsi de capter les dépendances temporelles locales avec un nombre de paramètres bien plus faible.

$$\text{Conv standard : } k \times C^2 \text{ paramètres} \quad -> \quad \text{Depthwise : } k \times C \text{ paramètres} \tag{1}$$

Pour $k = 31$ et $C = 256$: $2,031,616$ vs $7,936$ paramètres, soit une réduction d'un facteur d'environ 256.

Hyperparamètres du modèle Conformer :

Paramètre	**Valeur**
Dimension du modèle (d_{model})	256
Nombre de têtes d'attention	4
Couches encodeur Conformer	4
Couches décodeur Transformer	4
Dimension FFN	1 024
Taille noyau depthwise	31
Dropout	0.1
Taille du vocabulaire	1 000 tokens
Paramètres totaux	**10,8 M**

TABLE 1 – Hyperparamètres du modèle Conformer (Sprint 2)

Le passage du Transformer au Conformer ajoute environ 2,9 M de paramètres (7,9 M -> 10,8 M), principalement dus aux modules de convolution dans chaque couche encodeur.

2.2 - Beam Search avec pénalité de longueur

Décodage amélioré

Beam Search avec *length penalty* ($\alpha = 0.6$)
Remplace le décodage greedy (argmax simple) utilisé en Sprint 1

Problème du beam search sans pénalité :

Sans normalisation, le score d'une séquence est la somme de ses log-probabilités. Chaque token ajouté contribue négativement au score (les log-probs sont ≤ 0), ce qui favorise artificiellement les séquences courtes :

$$\text{score}(y_1, \ldots, y_U) = \sum_{u=1}^{U} \log P(y_u \mid y_{<u}, x) \tag{2}$$

Solution – normalisation par longueur :

$$\text{score_normalisé} = \frac{\text{score}}{U^{\alpha}}, \quad \alpha = 0.6 \tag{3}$$

La valeur $\alpha = 0.6$ est la valeur standard utilisée par Google (*Wu et al., 2016*) et reprise dans Whisper. Elle offre un bon compromis : $\alpha = 0$ revient au score brut (favorise les séquences courtes), $\alpha = 1$ divise par la longueur exacte (peut favoriser les séquences trop longues).

Algorithme Beam Search implémenté :

```python
def _beam_search(self, enc_out, enc_mask, B, device,
                 max_new_tokens, beam_size, length_penalty=0.6):
    for b in range(B):
        beams = [(0.0, [bos_id])]    # (score, sequence)
        completed = []

        for step in range(max_new_tokens):
            all_candidates = []
            for score, seq in beams:
                if seq[-1] == eos_id:
                    completed.append((score, seq))
                    continue
                # Predire le prochain token
                log_p = F.log_softmax(logits[0, -1, :], dim=-1)
                topk_lp, topk_ids = log_p.topk(beam_size)
                for lp, tid in zip(topk_lp, topk_ids):
                    all_candidates.append((score + lp, seq + [tid])
    )

            # Trier par score normalise
            all_candidates.sort(
                key=lambda x: x[0] / (len(x[1]) ** length_penalty),
                reverse=True
            )
            beams = all_candidates[:beam_size]
```

Listing 2 – Beam Search avec length penalty

2.4 - Entraînement et résultats préliminaires

Configuration d'entraînement :

Paramètre	Valeur
Optimiseur	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$)
Learning rate	3×10^{-4} avec warmup (2 000 steps)
Scheduler	Warmup + Cosine Decay
Batch size réel	4
Gradient accumulation	16 -> batch effectif : 64
FP16	Activé (`torch.amp`)
Label smoothing	0.1
Early stopping	5 époques

TABLE 2 – Configuration d'entraînement Sprint 2

Résultats préliminaires (entraînement en cours) :

Époque	Loss train	WER validation
1	4.47	134.08%
5	2.32	81.57%
10	1.84	50.26%
15	1.63	40.42%
17	1.60	37.78%
20	1.53	35.60%
22	1.50	29.52%
25	1.45	29.26%
Objectif		*<25%*

TABLE 3 – Progression de l'entraînement (en cours)

```
Époque 25/50 — 88.8 min
  Loss train : 1.6240
  Évaluation...
  REF : ils seraient également affectueux, intelligents et auraient une forte personnalité.
  HYP : il serait également affectueux, intelligents et aura une forte personnalité.

  REF : il aide le club à atteindre la de la super league.
  HYP : il est le club atteint de la de la superlig.

  REF : la fête patronale a lieu pour la saint laurent, agrémentée par bals, feu d'artifice.
  HYP : la fête patronale a lieu pour la saint-laurent, agrémentée par balle, folertifice.

  Loss val   : 1.4461
  WER val    : 29.26%
  Meilleur modèle sauvegardé — WER=29.26%
```

FIGURE 1 – Test du model en cours d'entrainemant (époque 25)

La décroissance de la loss reste régulière sur l'ensemble de l'entraînement, tandis que le WER diminue fortement au cours des premières époques avant de ralentir progressivement à partir de l'époque 20. Cette inflexion était prévisible, le modèle a désormais appris l'essentiel des correspondances acoustique-texte et n'effectue plus que des erreurs résiduelles, principalement liées à l'orthographe, à la grammaire, à la conjugaison, ainsi qu'à quelques rares hallucinations ponctuelles.
La phase actuelle d'entraînement correspond donc à un régime de raffinement, où les améliorations deviennent plus lentes mais portent sur des détails linguistiques de plus en plus subtils. Le WER, désormais proche de 30%, devrais continué de baisser (à voir dans la suite de l'entraînement), l'objectif fixé à moins de 25%.

Bilan du Sprint 2

Objectifs atteints

- ✓ Implémentation du ConformerConvModule from scratch (GLU, depthwise conv, BatchNorm, Swish)
- ✓ Implémentation de la ConformerLayer
- ✓ Migration de l'AudioEncoder : EncoderLayer -> ConformerLayer
- ✓ Beam search avec length penalty ($\alpha = 0.6$)
- ✓ Un premier test de pipeline d'inférence vidéo complète avec export `.srt` / `.vtt` / `.txt`
- ✓ Entraînement opérationnel sur RTX 3060 (WER ∼29% à époque 25)

Difficultés rencontrées

- **Incompatibilité des checkpoints** : les poids du Sprint 1 (7,9 M params, `EncoderLayer`) ne sont pas compatibles avec l'architecture Sprint 2 (10,8 M params, `ConformerLayer`), l'entraînement a dû repartir de zéro. (Je m'en doutait, mais c'est peu problématique car j'avait fait 3 époques seulement)
- **Sous-utilisation du GPU** : le batch size initial de 4 n'utilisait que 668 MB/6 144 MB de VRAM (11%) ; une augmentation à 16 a été identifiée pour accélérer l'entraînement.

Réflexion sur les résultats

L'architecture Conformer est bien adaptée à la reconnaissance vocale. Le module de convolution *depthwise* permet de mieux capter les motifs phonétiques locaux (transitions de phonèmes, durées), tandis que la self-attention conserve une bonne prise en compte du contexte global de la phrase. Cette combinaison est plus pertinente pour le signal audio qu'un Transformer seul.

L'ajout d'environ 2,9,M de paramètres reste modéré au regard du gain attendu sur le WER. L'utilisation de la convolution *depthwise separable*, issue notamment des travaux sur MobileNet, se montre ici efficace et adaptée au traitement audio.

Planification du Sprint final

1. **Inférence sur une vidéo**
   - Pipeline d'extraction audio depuis une vidéo (`ffmpeg`)
   - Génération et intégration de sous-titres au format SRT

Calendrier prévisionnel Sprint final :

Semaine	Tâche
10 – 17 avril	Entraînement de models (En cours)
17 – 22 avril	Benchmarking
22 avril – 3 mai	Pipeline vidéo + rapport final
Remise	**Dimanche 3 mai 2026**

Table 4 – Calendrier du Sprint final